

Progress Report: A PhiBE-Style Critic Target for Offline RL in Type-1 Diabetes Control

and Why It Requires a Non-Standard Optimizer

Hanzhang Yuan
Advisor: Prof. Yuhua Zhu

June 24, 2026

Abstract

We start from the offline reinforcement-learning (RL) baseline of Emerson et al. (2023) for blood-glucose control with TD3-BC, and make *a single modification*: we replace the critic’s regression *target*. Following the continuous-time value function, we Taylor-expand the expectation term inside the target. This expansion makes the value function V , its state gradient $\nabla_s V$, and its state Hessian $\nabla_s^2 V$ appear *simultaneously* in the objective. The resulting learning objective therefore has the form $f(V_\theta, \nabla_s V_\theta, \nabla_s^2 V_\theta)$, in sharp contrast to a standard RL objective $f(V_\theta)$ that depends on θ only through the network’s output value. We make the objective precise and then analyze why a standard RL optimizer cannot minimize it. Identifying a suitable optimizer for this derivative-bearing objective is the focus of the next stage.

1 Setup: the Emerson baseline and the single change we make

The Emerson et al. (2023) baseline trains a TD3-BC critic by regressing $Q_\theta(s, a)$ onto the standard discrete-time Bellman target

$$y^{\text{std}} = r + \gamma Q_{\theta^-}(s', \pi(s')), \quad (1)$$

where θ^- denotes the (frozen) target network. The *only* change we introduce is to the target y . Everything else in TD3-BC—the actor update, the behavior-cloning term, the replay buffer, and the target-network mechanism—is left untouched.

2 The continuous-time value function and its Taylor expansion

2.1 Starting point

We adopt the continuous-time value function

$$V(s_t) = r \Delta t + e^{-\beta \Delta t} \mathbb{E}[V(s_{t+1}) \mid s_t], \quad (2)$$

where $\beta > 0$ is a continuous-time decay rate (per unit time) and Δt is the sampling interval. (The state-action version, $Q(s_t, a_t) = r \Delta t + e^{-\beta \Delta t} \mathbb{E}[Q(s_{t+1}, a_{t+1}) \mid s_t, a_t]$, has the same structure.)

2.2 Taylor-expanding the expectation

Let $\Delta s := s_{t+1} - s_t$. Expanding $V(s_{t+1})$ about s_t to second order,

$$V(s_{t+1}) \approx V(s_t) + \Delta s^\top \nabla_s V(s_t) + \frac{1}{2} \Delta s^\top \nabla_s^2 V(s_t) \Delta s. \quad (3)$$

Taking the conditional expectation and introducing the (data-estimable) drift and diffusion statistics

$$\boldsymbol{\mu} := \frac{\mathbb{E}[\Delta s \mid s_t]}{\Delta t}, \quad \boldsymbol{\Sigma} := \frac{\mathbb{E}[\Delta s \Delta s^\top \mid s_t]}{\Delta t}, \quad (4)$$

substituting (3) into (2), expanding $e^{-\beta \Delta t} \approx 1 - \beta \Delta t$, and collecting the first-order terms in Δt , we obtain the core PhiBE relation

$$\boxed{\beta V(s) = r + \boldsymbol{\mu}^\top \nabla_s V(s) + \frac{1}{2} \text{tr}(\boldsymbol{\Sigma} \nabla_s^2 V(s)).} \quad (5)$$

Equation (5) is the first point at which the three objects V , $\nabla_s V$, and $\nabla_s^2 V$ appear *together*.

3 The learning objective

Define the per-state PhiBE residual of a parametric value function V_θ :

$$\mathcal{R}_\theta(s) := \beta V_\theta(s) - r - \boldsymbol{\mu}^\top \nabla_s V_\theta(s) - \frac{1}{2} \text{tr}(\boldsymbol{\Sigma} \nabla_s^2 V_\theta(s)). \quad (6)$$

The objective to be minimized is the mean-squared residual over the data distribution $\mathcal{D}$:

$$\boxed{f(\theta) = \mathbb{E}_{s \sim \mathcal{D}}[\mathcal{R}_\theta(s)^2] = \mathbb{E}_s[(\beta V_\theta - r - \boldsymbol{\mu}^\top \nabla_s V_\theta - \frac{1}{2} \text{tr}(\boldsymbol{\Sigma} \nabla_s^2 V_\theta))^2].} \quad (7)$$

The three terms. Writing d for the state dimension:

Term	Object	Shape
First	$V_\theta(s)$	scalar (coefficient β)
Second	$\nabla_s V_\theta(s)$	vector in $\mathbb{R}^d$ (contracted with $\boldsymbol{\mu}$)
Third	$\nabla_s^2 V_\theta(s)$	matrix in $\mathbb{R}^{d \times d}$ (trace against $\boldsymbol{\Sigma}$)

Crucially, $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are estimated from data and carry *no* dependence on θ ; the only θ -dependent objects are V_θ , $\nabla_s V_\theta$, and $\nabla_s^2 V_\theta$.

Contrast with standard RL. A standard RL objective has the form $f(V_\theta(s)) = \mathbb{E}[(V_\theta(s) - y)^2]$, where the target y is treated as a constant; it depends on θ only through the network’s *output value*. Our objective (7) has the form

$$\underbrace{f(V_\theta)}_{\text{standard RL}} \longrightarrow \underbrace{f(V_\theta, \nabla_s V_\theta, \nabla_s^2 V_\theta)}_{\text{PhiBE}}, \quad (8)$$

i.e. the loss depends on θ through the output value *and* its first and second derivatives with respect to the input.

4 Why a standard RL optimizer cannot minimize (7)

Standard RL optimization (SGD/Adam with semi-gradient bootstrapping) is built on the assumption that the loss depends on θ *only* through the output value. Objective (7) violates this assumption, and three distinct difficulties follow.

Difficulty 1: mixed parameter–input higher-order derivatives

Minimizing f requires $\nabla_\theta f$. By the chain rule,

$$\nabla_\theta f = 2 \mathbb{E}[\mathcal{R}_\theta(s) \nabla_\theta \mathcal{R}_\theta(s)], \quad (9)$$

and $\nabla_\theta \mathcal{R}_\theta$ contains

$$\nabla_\theta(\nabla_s V_\theta) \quad \text{and} \quad \nabla_\theta(\nabla_s^2 V_\theta). \quad (10)$$

A standard critic update needs only $\nabla_\theta V_\theta$ —a single backward pass. Here we additionally need $\nabla_\theta \nabla_s V_\theta$ (a mixed second derivative: differentiate once w.r.t. the input, once w.r.t. the parameters) and $\nabla_\theta \nabla_s^2 V_\theta$ (a mixed third derivative). These objects do not appear anywhere in a standard RL pipeline; computing them requires nested differentiation through the network at every step, which is expensive and numerically delicate.

Difficulty 2: the derivative terms are uncontrolled

A standard objective constrains the network’s *output value* directly. But nothing in (7) directly constrains the magnitude of $\nabla_s V_\theta$ (the input slope) or $\nabla_s^2 V_\theta$ (the input curvature). A neural network can produce reasonable values while having arbitrarily steep slopes. Consequently the term $\boldsymbol{\mu}^\top \nabla_s V_\theta$ entering the bootstrapped target can grow without bound during training, driving the optimization to diverge. A standard optimizer presumes the loss terms are well behaved; that presumption fails here.

Difficulty 3: the second-order term is ill-posed for common networks

The third term depends entirely on the second-order differentiability of the network:

- For ReLU networks, $\nabla_s^2 V_\theta$ is (almost everywhere) zero, so the third term vanishes identically and the expansion collapses to first order, discarding the diffusion information the second-order term is meant to carry.
- For smooth activations (tanh/GELU), $\nabla_s^2 V_\theta$ exists but is typically oscillatory and difficult to control in magnitude, making the optimization less stable.

Summary of the difficulty

The root cause is structural: a standard optimizer assumes the loss depends on θ *only through the output value*, whereas (7) depends on θ through the *value, the input gradient, and the input Hessian*. This forces mixed higher-order derivatives (Difficulty 1), exposes unconstrained quantities to the bootstrapping dynamics (Difficulty 2), and renders the second-order term ill-posed under common network choices (Difficulty 3).

5 Next step

The objective (7) is now stated precisely, and the reasons a standard RL optimizer fails to minimize it are understood. The next stage is to identify a suitable optimization method that can minimize an objective depending jointly on V_θ , $\nabla_s V_\theta$, and $\nabla_s^2 V_\theta$. We would value the advisor’s guidance on what form this optimizer should take before committing to an implementation.